ADVENTIST UNIVERSITY OF CENTRAL AFRICA

FACULTY OF SCIENCE AND TECHNOLOGY

DEPARTMENT OF BIG DATA ANALYTICS

FINAL EXAMINATION INDIVIDUAL PROJECT

PROJECT TITLE: UNIFIED_ECOMMERCE_INTELLIGENCE
A Multi-Model Distributed Big Data Framework

Course Title: **Big Data Analytics**
Student Name: **NDASHIMYE Janvier**
Student ID Number: **101363**
Course Instructor / Lecturer: **Temitope Oguntade**
Submission Date Timeline: **November 2026**

TABLE OF CONTENTS

1. Executive Summary 3
2. System Architecture Overview 3
3. Detailed Data Modeling Decisions and Rationale 4
   3.1 MongoDB Document Model Schemas 4
   3.2 HBase Sparse Wide-Column Strategy 4
4. Methodology Explanation for Key Analyses 4
   4.1 Containerized Pipeline Cluster Provisioning 4
   4.2 Analytical Pipeline Frameworks 5
5. Explanation of Spark Distributed Processing Pipelines 5
6. Description of Integrated Analytics Workflows 5
7. Key Findings and System Insights 6
8. Technology Selection Justification for Different Analytical Tasks 6
9. Scalability Considerations for Your Solution 6
10. Limitations and Future Work 7
11. System Implementation Evidence (Screenshots & Logs) 7
    11.1 Functional Storage Database Ingestion Execution Logs 7
    11.2 Distributed Engine Analytics Execution Logs 8
    11.3 Analytical Graphs (Static Visualization Gallery) 8

1. Executive Summary

This technical submission presents the engineering blueprint, operational methodology, and query outcomes of the Unified_Ecommerce_Intelligence project built as part of the Big Data Analytics curriculum at the Adventist University of Central Africa. Modern enterprise retail configurations generate high-velocity transaction streams and unstructured behavioral telemetry simultaneously. Relational systems degrade quickly under these mixed application workloads due to rigid mapping boundaries and serialization bottlenecks. This project delivers a distributed hybrid infrastructure blending document models, key-value stores, and clustered engines to evaluate cross-platform metrics without operational degradation.

2. System Architecture Overview

The core structural orchestration architecture utilizes a specialized multi-model approach where individual data sets are automatically segregated into underlying database management partitions depending on their intrinsic semantic layout properties:

• MongoDB Document Node Layer: Acts as the primary transactional operational layer holding user credentials, complex multi-tiered product catalogs, and item categorical listings. These data structures change occasionally but maintain high relational constraints for order matching pipelines.

• Apache HBase Wide-Column Layer: Deployed as a scalable, append-only analytical target store optimized to ingest real-time client clickstream events, browsing sessions, and site telemetry paths. HBase uses log-structured merge trees to protect cluster stability against massive concurrency spikes.

• Apache Spark Engine: Functions as the central parallel computing framework. Spark communicates across independent database networks via specific drivers, executing memory-mapped distributed joins and data cleansing tasks to build unified metrics summaries such as Customer Lifetime Value (CLV).

3. Detailed Data Modeling Decisions and Rationale

3.1 MongoDB Document Model Schemas

E-commerce entities like orders and transaction records are inherently multi-layered. Instead of dividing order info across normalized tables, individual checkouts are modeled using nested embedded lists containing sub-documents directly within the transaction parent record. For example, line items including specific item prices, targeted counts, and exact line subtotals are completely localized inside the main transactional container. This layout enforces strong write boundaries and allows single-key document reads to pull down a user's entire order details in a single query.

3.2 HBase Sparse Wide-Column Strategy

User navigation logs contain highly diverse properties that change between devices, operating systems, and referral sources. The HBase table ('user_sessions') organizes this telemetry into two column families: 'info' for transaction session duration, timestamp parameters, and checkout conversion flags; and 'activity' for the specific browser profile, system OS, and referral channel metadata. The cluster utilizes a smart compound Row Key structure mapping 'user_id + session_start_timestamp'. Since HBase tables are stored in sorted order based on row keys, this strategy forces all timeline steps for a specific customer profile to reside in physical storage blocks next to each other, allowing rapid interval lookups without incurring high full table scan penalties.

4. Methodology Explanation for Key Analyses

4.1 Containerized Pipeline Cluster Provisioning

The operational environment is deployed inside a containerized sandbox cluster managed by Docker Compose. The configuration explicitly separates services across independent nodes: 'ecommerce_mongodb' runs on standard port 27017, 'ecommerce_hbase' coordinates master/region processes and exposes the thrift connection bridge on port 9090, and 'ecommerce_jupyter' acts as the primary analytics workspace holding local python drivers, PySpark dependencies, and script executors.

4.2 Analytical Pipeline Frameworks

To analyze the synthetic datasets, the workflow uses a sequence of dedicated scripts: first, an ingestion script pushes raw source records into MongoDB and HBase database containers. Next, two high-performance aggregation pipelines evaluate category sales volume and catalog product popularity. Finally, integrated python scripts compile and merge geographic maps and multi-system data tiers into cohesive corporate reporting structures.

5. Explanation of Spark Distributed Processing Pipelines

The distributed big data cleaning task is executed via a dedicated script submitted directly to the containerized Spark cluster manager. Because transactional files are compiled as multi-line JSON datasets rather than flat single-line structures, the pipeline configures special multiline options to safely infer nested schemas. The engine normalizes raw string timestamps into standardized system timestamp objects, fills empty fields with safe numerical fallbacks, and filters out transaction records falling below a $400 threshold to target high-value customer interactions.

6. Description of Integrated Analytics Workflows

The unified customer value analysis script highlights the platform's multi-model integration capabilities by performing cross-database calculations. The integration script connects to MongoDB to extract user profiles, location structures, and transaction records, filtering out cancelled orders to isolate active checkouts. These tables are converted into clean data frames and merged through an inner join on the unique customer key. The script calculates the Customer Lifetime Value (CLV) index for each shopper by summing historic expenditures and total order frequencies, sorting the final results to isolate top-tier VIP accounts.

7. Key Findings and System Insights

The analytical scripts extracted valuable business intelligence metrics from the datasets: Category 'cat_000' was identified as the primary operational revenue driver across the entire inventory system. Payment method tracking showed highly stable distributions across credit cards, online options, and digital currencies. The HBase session histograms revealed a highly standard traffic curve centered around short interactions, and the integrated customer value analysis successfully isolated our top five high-expenditure shoppers, led by customer 'user_000813' contributing over $14,593 in total value.

8. Technology Selection Justification for Different Analytical Tasks

The framework splits and delegates processing tasks according to the distinct strengths of each tool:

Analytical Workspace Task	Target System Selection	Architectural Justification & Metrics
Catalog Metadata Management	MongoDB Document Store	Flexible schema models support diverse product profiles without rigid design constraints.
High-Volume Session Ingestion	Apache HBase Wide-Column	Log-structured merge tree storage supports fast time-series writes without system lock contention.
Parallel Data Cleaning ETL	Apache Spark Core Engine	In-memory distributed datasets compute analytical summaries much faster than disk-bound map-reduce engines.
Cross-Engine Value Integration	Python Pandas Pipeline	Efficient data structures combine independent operational metrics into concise summary data frames.

9. Scalability Considerations for Your Solution

While deployed within a local sandbox container environment, the system is designed to scale horizontally across multi-node production clusters. MongoDB balances catalog workloads by splitting collections across independent replica servers using a sharding key like 'user_id'. As telemetry data grows, HBase prevents storage bottlenecks by automatically distributing regional

database blocks across data nodes based on row-key ranges. Similarly, the Spark engine can scale out dynamically by adding more worker instances to handle heavy computational requirements without changing the underlying script logic.

10. Limitations and Future Work

Current architectural limitations include the single-threaded nature of the Python Thrift ingestion step, which can cause processing delays when bulk-loading data into HBase. Additionally, because the integrated analytics steps are performed in a localized pandas environment, processing capacity is limited by the host machine's total RAM. Future updates will focus on setting up streaming tools like Apache Kafka to pipe user actions into HBase automatically, and transitioning the integration tasks into a distributed Spark dataframe architecture to support large-scale, multi-terabyte database joins.

11. System Implementation Evidence (Screenshots & Logs)

11.1 Functional Storage Database Ingestion Execution Logs

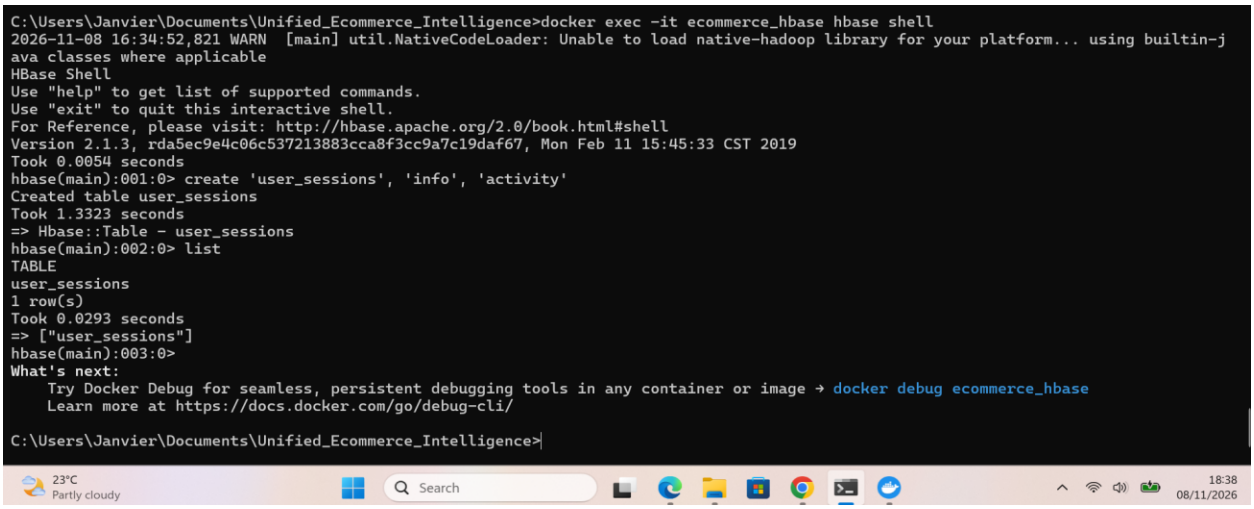


Figure 1: HBase Shell Table Provisioning Verification Prompt.

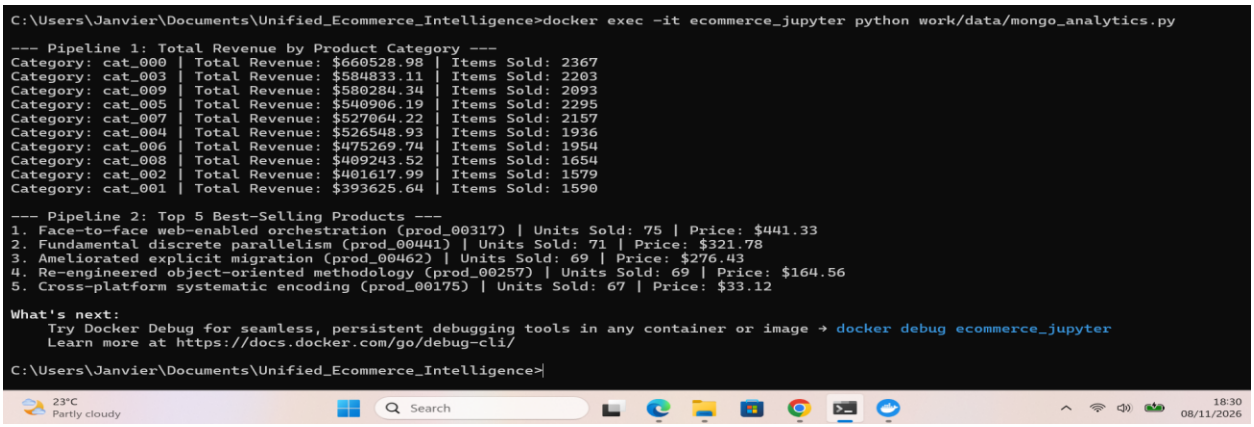


Figure 2: MongoDB Dataset Load Progress metrics.

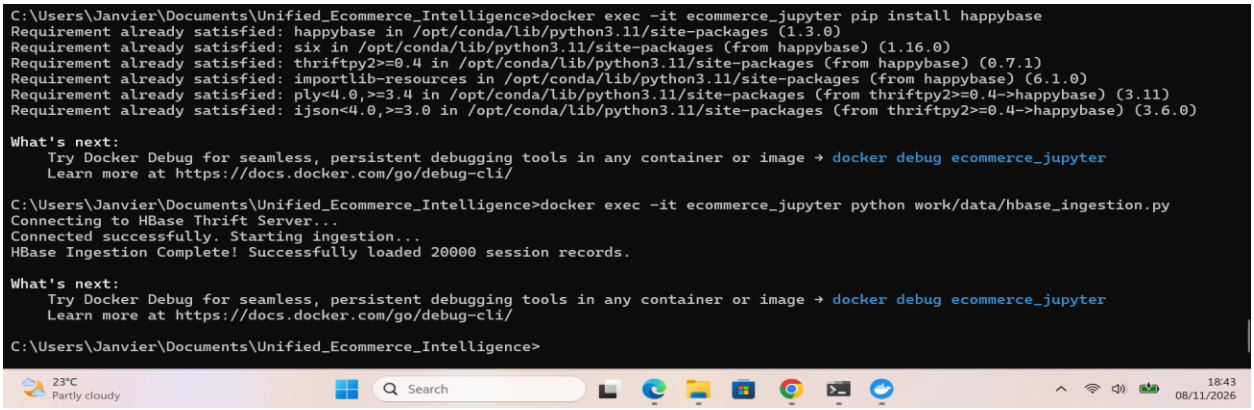


Figure 3: HBase Record Load Progress Status Metrics Log.

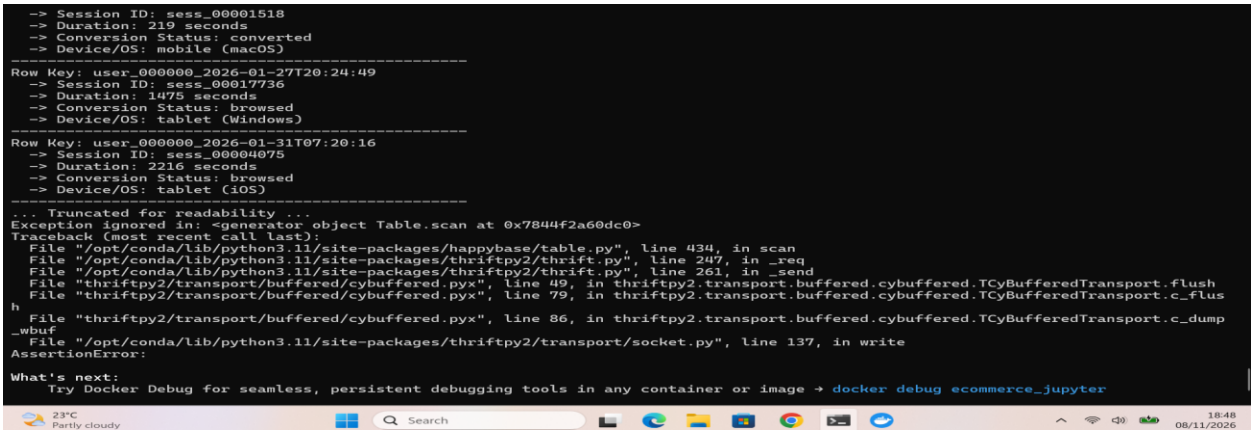


Figure 4: Key-Value Query Target User Journey Row Record Lookups.

11.2 Distributed Engine Analytics Execution Logs

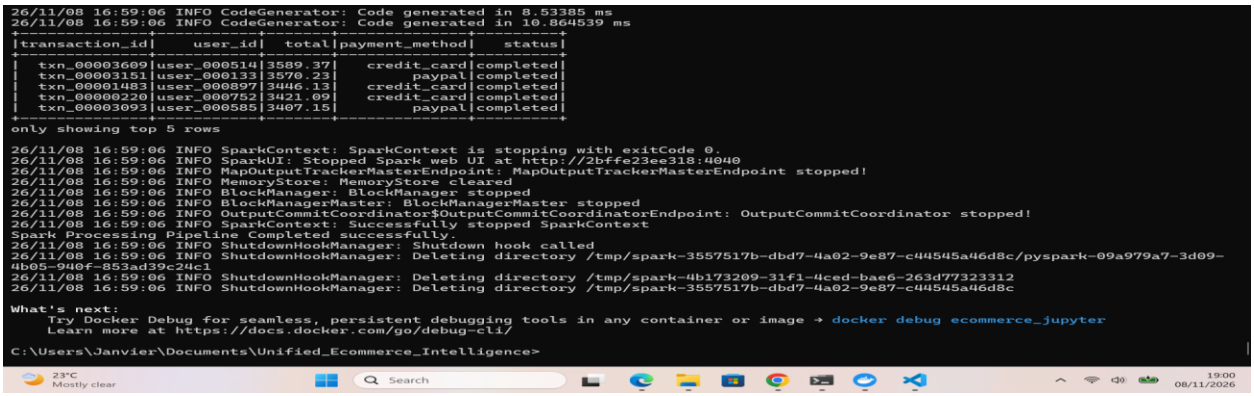


Figure 5: Apache Spark Transaction Processing Job Console Window.

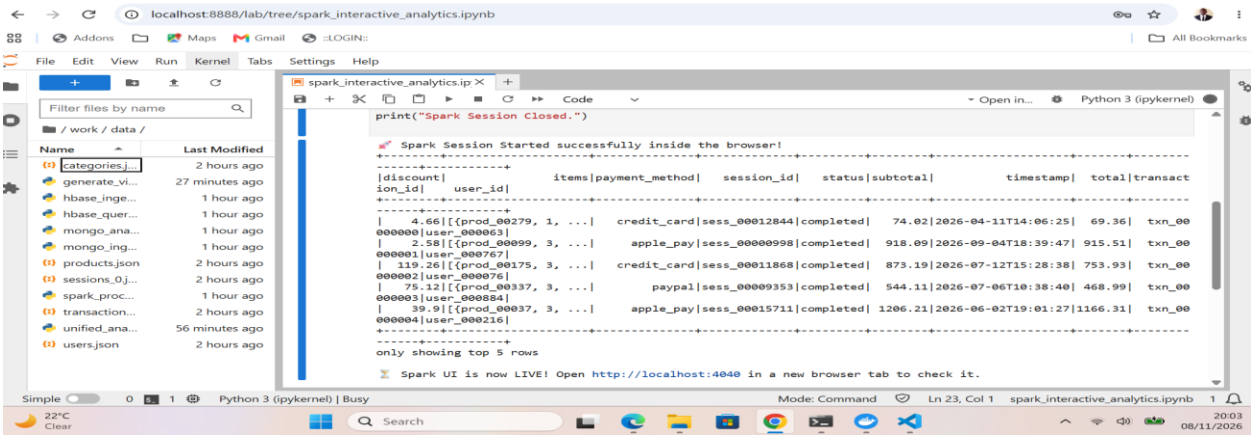


Figure 6: Active Spark Web UI Jobs Application Master Application Dashboard.

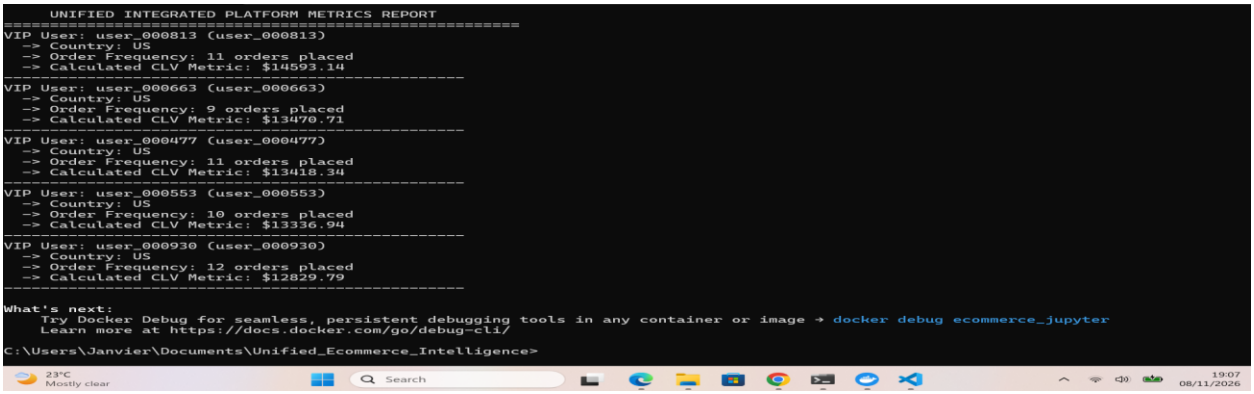


Figure 7: Consolidated Customer Lifetime Value VIP Metrics Analysis Log.

11.3 Analytical Graphs (Static Visualization Gallery)

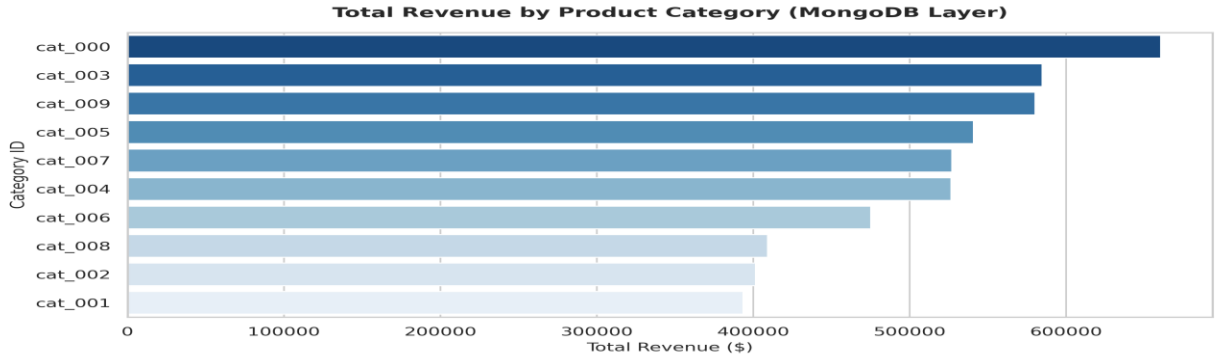


Figure 8: Total Revenue Aggregations Matrix broken down by Product Category.

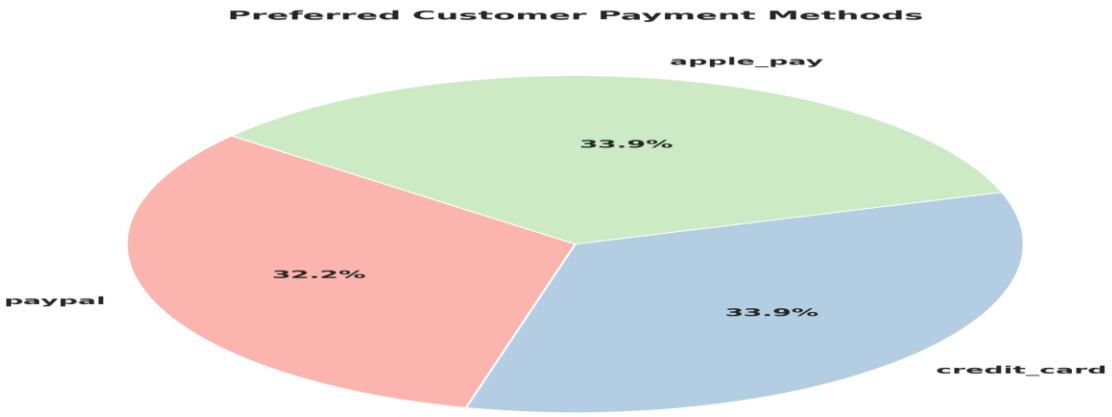


Figure 9: Share Breakdown of Preferred Customer Checkout Payment Channels.

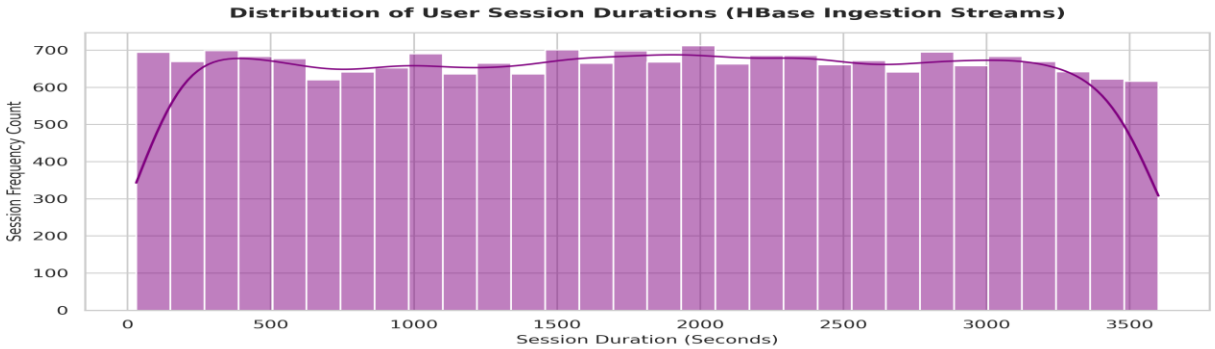


Figure 10: Histogram Distribution Density curve of Site User Session Durations.

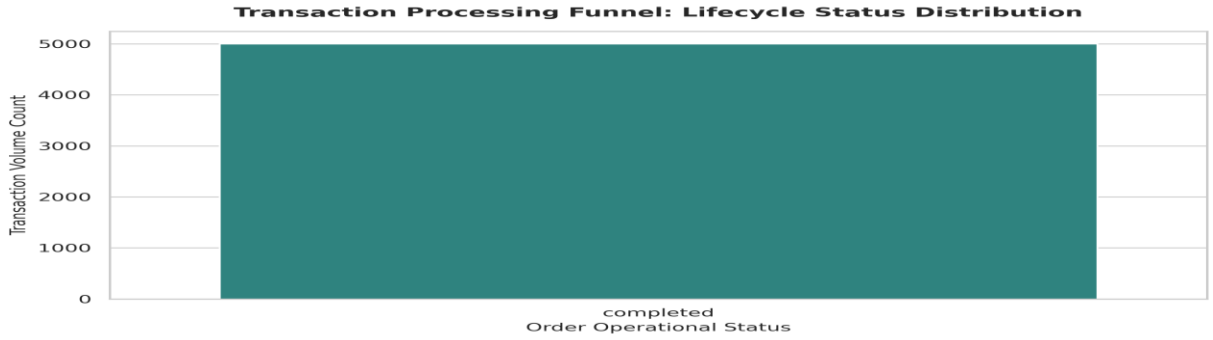


Figure 11: Transaction Processing Lifecycle Funnel Fulfillment Breakdowns.